

1 Project Report: Switch Transformer - ROAST

The objective of this study is to build upon the initial research by enhancing the model compression technique for Switch Transformer using ROAST hashing. We aim to surpass the results achieved in the original study by optimizing the compression process through ROAST, potentially achieving even greater compression ratios while maintaining model quality and performance.

2 Literature Review

2.1 Switch Transformer

The Switch Transformer, belonging to the mixture of experts family of models, operates by selecting different sets of parameters for each incoming example. These parameters are organized into separate clusters referred to as experts. During each prediction, only a subset of these experts is utilized. Figure 1 gives a visual example of the switch component of the network.

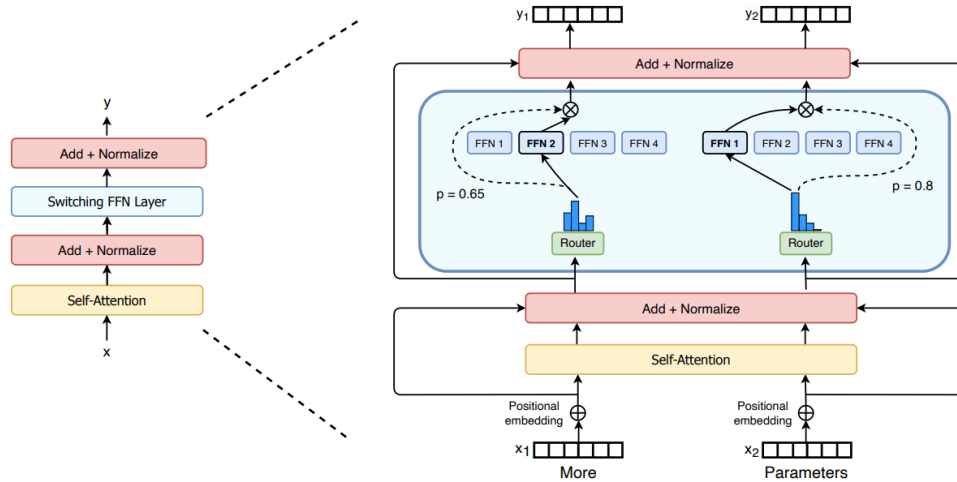


Figure 1: Example of the Switch routing layer.

This approach results in a model with sparse activations and an extensive number of parameters, while maintaining a consistent computational cost due to a constant number of activated neurons. While Mixture of Experts (MoE) models have shown remarkable success, their widespread adoption has been impeded by factors such as complexity, communication overhead, and training instability [2].

2.2 Distillation Strategy

Google's research team has explored the distillation of the Switch-Base model into a *dense* architecture, specifically T5-Base. This endeavor persevered 37% of the teacher model quality while achieving an 82% compression rate. At the extreme end, with a 99% compression rate,

they managed to preserve 28% of the teacher’s model quality [2]. We, on the other hand, are trying to compress the model using the same *sparse* architecture.

2.3 ROAST Hashing

In the realm of model compression, the Random Operation Access Specific Tile (ROAST) hashing method, as proposed by [1], presents a promising avenue. ROAST efficiently collapses model parameters by grouping them through a lightweight mapping mechanism. This approach has demonstrated the ability to achieve comparable or superior accuracy while compressing models by a factor of 10 to 1000 times.

3 Bold Statement

While Google has not done compression of the Switch Transformer sparse architecture to a smaller similar architecture, this project will aim to achieve that.

4 Experiment Setup

4.1 Data

The experiment involves recreating the Switch Transformer and T5 Google models in their original settings to establish a baseline for comparing the compression rate. The original paper trained these models on the extensive C4 dataset, which exceeds 300GB. However, for the purpose of this study, we initiated training on a similar dataset, Code Search Net, that is consistent with code from various different languages [3]. Due to computing limitations, the training dataset consisted of 2K examples of the given dataset. We figured that if we can show the results with a small amount of that, that will extrapolate to more data.

4.2 Baseline Model

To initialize the models, we utilized a configuration provided by the Google Switch Base 8 model, which is the smallest version, with 620M parameters, of the Switch Transformer available on Hugging Face.

The primary objective task pertains to a variation of the masked language model task that involves replacing spans of text [4]. This process handles consecutive spans of dropped-out tokens by substituting them with a unique sentinel token. These sentinel tokens are introduced as entirely new entities within the vocabulary and do not represent any specific word or word piece. They serve as markers to indicate the boundaries of the dropped-out spans within the input sequence. Additionally, a final sentinel token signifies the conclusion of the target sequence. An illustrative example can be seen in Figure 2. The advantage of this objective is that the model only predicts (outputs) the masked words rather than recreating the whole text. This saves training time and improves accuracy.

To facilitate this, a new collator was created, which is a replica of the one used for the T5 model.

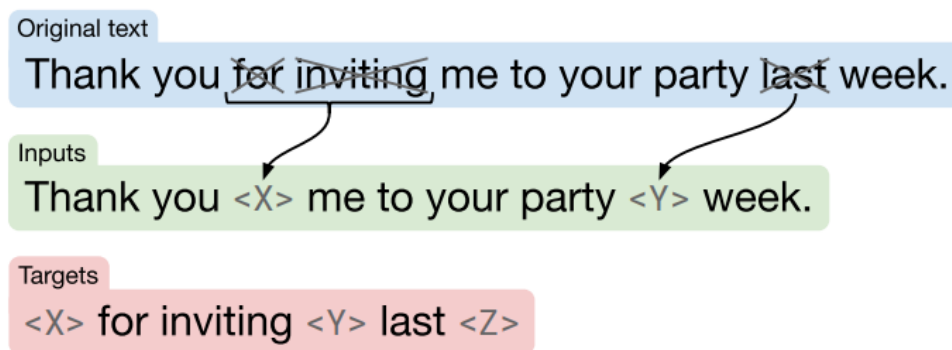


Figure 2: Example of the Span Task.

After establishing the baseline configuration, we embarked on the ROAST process for the baseline model, exploring various sparsity levels. Specifically, we examined sparsity values of 0.1, 0.2, 0.3, 0.4, and 0.5. Given the focus of this study on large language models, we adopted perplexity as our primary evaluation metric.

5 Training

The baseline model was trained for 100 epochs with a batch size of 32 and a gradient accumulation strategy with 4 steps.

Subsequently, we sought to train the compressed version of the model under equivalent conditions to facilitate a meaningful comparison. This compressed model underwent 100 epochs of training, albeit with adjustments to the training parameters. We employed a reduced batch size of 16 and implemented an 8-step gradient accumulation scheme. This amounts to the same number of steps in both the baseline and compressed model scenarios.

Consistent with the overarching goals of this research, perplexity was employed as the primary evaluation metric, given its relevance for gauging language model performance.

The models were trained on a single GPU with 32GB of RAM.

6 Results

The results of our experiments with compressed models have unveiled promising findings. Notably, we observed that the compressed models demonstrated perplexity values that were close to the full, non-compressed model. This remarkable outcome underscores the significant potential of the ROAST technique in preserving crucial information while achieving substantial reductions in model size.

In Table 1, we present a concise summary of these results, highlighting the tradeoff between model size and accuracy. The baseline perplexity score serves as a reference point, with subsequent rows demonstrating how different sparsity levels influence model performance. This analysis provides valuable insights for practitioners facing memory constraints,

as it allows them to select a smaller model that matches the original’s performance.

Sparsity Value	Perplexity ↓
Baseline	0.7114
0.1	2.1708
0.2	2.0265
0.3	1.9289
0.4	1.9185
0.5	1.8914

Table 1: Perplexity Results for Different Sparsity Values

Furthermore, in Figure 3, we visualize the model training progress over the training steps. This graphical representation allows us to observe a noteworthy phenomenon known as ”Double Descent.”

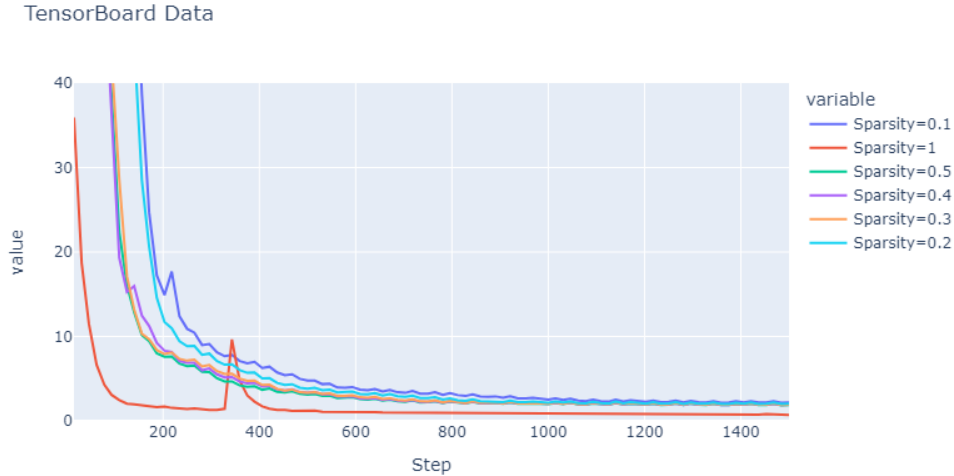


Figure 3: Training Loss Progression for All Models.

The phenomenon of Double Descent is explored in more detail in Figure 4, where we examine the behavior of the baseline model. This graph illustrates the intriguing Double Descent curve, a phenomenon in which model performance initially worsens as model complexity increases, improves with a moderate level of complexity, and then degrades again as complexity continues to increase.

In summary, our research not only showcases the potential of ROAST in achieving efficient model compression but also sheds light on the intriguing Double Descent phenomenon in the context of large language models. These findings hold significant implications for practitioners seeking to strike a balance between model size and performance in resource-constrained environments.

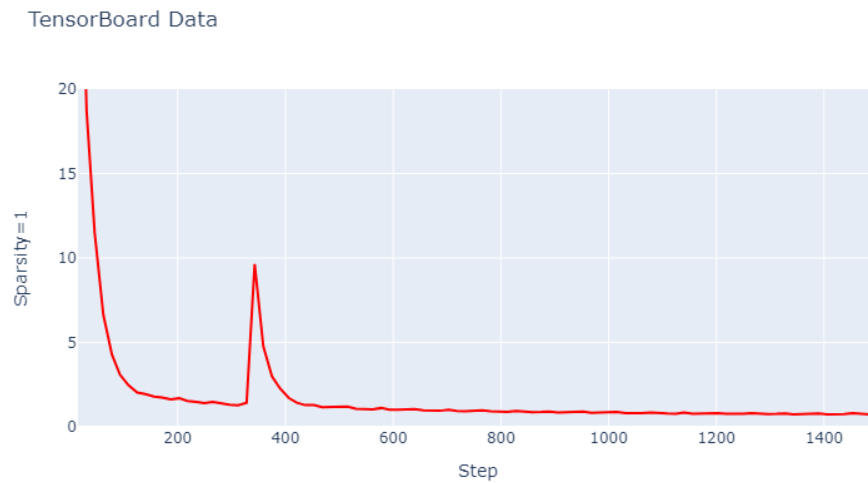


Figure 4: Double Descent Curve for the Baseline Model.

References

- [1] Aditya Desai, Keren Zhou, and Anshumali Shrivastava. *Efficient model compression with Random Operation Access Specific Tile (ROAST) hashing*. 2022. arXiv: [2207.10702](#) [cs.LG].
- [2] William Fedus, Barret Zoph, and Noam Shazeer. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. 2022. arXiv: [2101.03961](#) [cs.LG].
- [3] Hamel Husain et al. “CodeSearchNet challenge: Evaluating the state of semantic code search”. In: *arXiv preprint arXiv:1909.09436* (2019).
- [4] Colin Raffel et al. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”. In: *CoRR* abs/1910.10683 (2019). arXiv: [1910.10683](#). URL: <http://arxiv.org/abs/1910.10683>.